

Are low-rank safety updates in LLMs less robust?

Adam Harrison



Master of Science
Artificial Intelligence
School of Informatics
University of Edinburgh
2026

Abstract

Open-weight LLMs offer better research access and the potential for broader, more democratic control over the technology. They do, however, pose additional risks over closed-source alternatives, especially the ease of down-stream fine-tuning, enabling the removal of safety mechanisms. Previous studies have suggested that the low-rankedness of safety updates is a cause of the fragility of safety mechanisms even in the case of benign fine-tuning. This dissertation investigates the hypothesis by safety tuning a pretrained model using 4 different combinations of training methods (SFT vs DPO) and optimisers (SGD vs Adam). The fine-tunes are then attacked in a benign fine-tuning scenario using a utility dataset. Pre- and post-attack safety, as well as the low-rankedness of the initial safety update, are measured.

We find that Adam does indeed result in a more low-rank update than SGD. Moreover, for DPO runs, lower rank safety updates do result in more brittle safety mechanism that degrade more after benign fine-tuning.

Research Ethics Approval

This project was planned in accordance with the Informatics Research Ethics policy. It did not involve any aspects that required approval from the Informatics Research Ethics committee.

Declaration

I declare that this thesis was composed by myself, that the work contained herein is my own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

(Adam Harrison)

Generative AI use Declaration

Generative AI was used during this project to assist with writing and debugging code for experiments and analysis, to help discuss and troubleshoot experimental design and implementation choices during the research process, and to edit and proofread text after it had already been drafted (for example, suggesting improvements to clarity, phrasing, and structure). Generative AI was not used to draft or write any of the text of this dissertation (other than this declaration); all other writing is my own.

Acknowledgements

Thanks, Heather for putting up with me for these last 3 months of writing the dissertation, and helping me stay more or less put-together. Thanks to Ivan and Neil for advice, reviews and discussions about safety of open-weights and supervising the project.

Table of Contents

1	Introduction	1
1.1	Motivation	1
1.2	Research Question	2
1.3	Contribution	3
2	Background	4
2.1	Fine-tuning	4
2.1.1	Methods	4
2.1.2	Optimisers	5
2.1.3	PEFT/LoRA	6
2.2	The closed-open spectrum of LLMs	7
2.3	Fine-tuning attacks and defences on LLMs	8
2.3.1	Defences	9
3	Methodology	11
3.1	Datasets	11
3.1.1	Aligning dataset	12
3.1.2	Attack dataset	13
3.2	Aligning	13
3.3	Safety update measurement	14
3.3.1	Activation extraction and measurement	15
3.3.2	Low-rank measurement	16
3.4	Safety Measurement	17
3.5	Attack	18
4	Results	19
4.1	Interpreting a Lorenz Curve	19
4.2	What causes the low-rankedness of a safety update?	20

4.3	ASR through safety training	22
4.4	Does low-rankedness of the safety update increase susceptibility to downstream fine-tuning?	24
5	Conclusion	27
5.1	Limitations & Future Work	28
	Bibliography	30

Chapter 1

Introduction

1.1 Motivation

Progress in the capabilities of LLMs has been rapid. Recently, the cyber-capabilities of the latest class of models have attracted a great deal of attention, including Claude Mythos 5 [1], which has the capability of discovering and exploiting cybersecurity vulnerabilities. One of them was a flaw in the OpenBSD operating system, which has remained undiscovered for 27 years. More recently, and worryingly, OpenAI's GPT-5.6 Sol hacked Hugging Face during a cyber evaluation [2], which was an example of the system escaping containment and causing real-world harm. Both cases illustrate the potential dangers of model deployment, though notably both models are closed models. If and when a deployment is later deemed too dangerous, it can be later rolled back, as was the case for Claude Fable 5, following export controls instituted by the US Government [3]. This control over models affords some safety, but also grants whoever wields it significant power: the unilateral ability to remove other actors' access to the model.

Open-weight models are behind the frontier, but not significantly, as the recent release of Kimi-K3 has shown [4, 5]. These models come with benefits, such as removing power and control from a narrow set of actors, as well as permitting open research on them. However, they come with one major downside, namely the lack of control of downstream risks. If the Kimi model had the same cyber capabilities exhibited by Claude Mythos and GPT-5.6 Sol, this capability, once released, could not be taken back. Once published, it could then be re-hosted many times, making it impossible to prevent dangerous actors from obtaining it [6].

Enforcing the safety of open-weight models is also made more difficult by the lack

of enforceable input/output monitors that do not rely on the intrinsic refusal behaviour of the model, but instead provide an additional layer of defence [6]. The final nail in the coffin is that even if a model is released with sufficient safeguards including robust refusal mechanisms, these can be easily degraded using fine-tuning, which in some cases could successfully disable the refusal mechanism for less than \$1 of compute, using publicly available libraries [7]. The fine-tuning does not have to be a deliberate attempt to remove the safety mechanism; it can even occur when fine-tuning with an instruction-following dataset like Alpaca [8]. The customisability of open-weight models is one of their great advantages. Modifications that unintentionally lead to safety degradation are a concern for anyone trying to modify a model to suit their needs. The fragility of the refusal mechanisms of open-weight models under fine-tuning is of particular concern.

This dissertation focuses on the latter risk: safety degradation from benign fine-tuning. In particular, we build upon [9, 7], which observe that safety updates introduced by fine-tuning are low-rank in both the weight and activation space. The latter [7], stipulates that the brittleness of safety fine-tuning may be caused by the low-rankedness of the safety update. We test this hypothesis, by performing a series of tests aimed at determining the relationship between the low-rankedness of an update and its brittleness. Additionally, we explore what causes the low-rankedness of a safety update. We investigate the effect of the training method comparing DPO [10], where we use a contrastive objective to train the model to prefer a safer response over a less safe one, to SFT [11], where the model imitates the provided samples. We also investigate the effect of the optimiser used, comparing the widely used Adam optimiser [12] to its predecessor, Stochastic Gradient Descent.

1.2 Research Question

We have two primary research questions:

1. Does the low-rankedness of safety updates cause the resulting safety mechanism to be more brittle and easier to be fine-tuned away?
2. What causes the low-rank nature of safety updates?

Answering these could help find ways to produce open weight models more robust to fine-tuning attacks. Low-rank safety updates may produce more brittle refusal mechanisms for two reasons: fewer ranks need to be adjusted in the weight matrices to

remove them, and the entire safety mechanism is more likely to rely on a few critical links that are easier to isolate and destroy [13]. To answer the second question we focus on investigating the impact of training methods (i.e. SFT versus DPO) and optimizers on the low-rankedness of a model. We hypothesise that DPO updates might be more low-rank than SFT updates. The DPO loss is defined through preferences between a chosen and rejected sample or response, indicating pull as well as push directions, whereas supervised fine-tuning only indicates pull directions. The choice of optimizer also plays a role in the rankedness of the updates. Adam normalises each parameter's update based on its gradient history, giving parameters with small gradients larger updates and vice-versa. Stochastic gradient descent has no such correction, so it is more sensitive to the shape of the loss landscape resulting in much larger updates in steeper directions and smaller updates in shallow ones. Thus we expect SGD updates to be more low-rank than Adam updates.

1.3 Contribution

Our contributions consist of:

- Refining the SafeRLHF dataset to obtain comparable training baselines for SFT and DPO training.
- Measuring the low-rankedness of a safety update by measuring the Gini index of the singular value spectra of its matrices in both the parameter and activation space.
- Empirically investigating the causes of low-rankedness of safety updates.
- Providing access to our code, datasets and models, thus enabling reproducibility and furthering the goals of open research.

We begin with background on fine-tuning methods and optimisers, followed by recent advances in open-weight attacks and defences, in Chapter 2. Next, in Chapter 3 we outline our experimental pipeline and the techniques used to produce comparable fine-tunes of models under different configurations, and define and justify the metrics used to measure the safety of models and the low-rankedness of the safety update. In Chapter 4, we present our results, focusing on answering our two research questions. Finally, in Chapter 5 we conclude with a discussion of the relevance of our results, the limitations of our study, and potential future work.

Chapter 2

Background

2.1 Fine-tuning

2.1.1 Methods

Training methods describe how we define objective functions and process data to aid learning of the LLM.

Supervised fine-tuning SFT is mechanically very similar to pre-training, with similar loss functions, but is usually performed with a smaller, more heavily curated dataset to increase performance on specific tasks. The aim is to have the model imitate the training data. When data generated by another LLM is used, the data-generating LLM is called a teacher and the trained LLM is called a student [14]. See Figure 2.1 for an example of the training data used to fine-tune a model with SFT.

Direct Preference Optimisation DPO is an algorithm designed to leverage the success of Reinforcement Learning from Human Feedback (RLHF [16]) without the difficulty of Reinforcement Learning and the overhead of training additional reward models and sampling from the original model [10]. The technique requires preference data: pairs of text with a shared prompt and different completions, one 'chosen', one 'rejected'. See an example in Figure 2.2. These preferences are usually collected from human annotators, or increasingly from other LLMs [17]. The training objective scores the two responses and redistributes the probability mass from the 'rejected' response to the 'chosen' one, thus making the model more likely to produce answers more like the first and less like the latter.

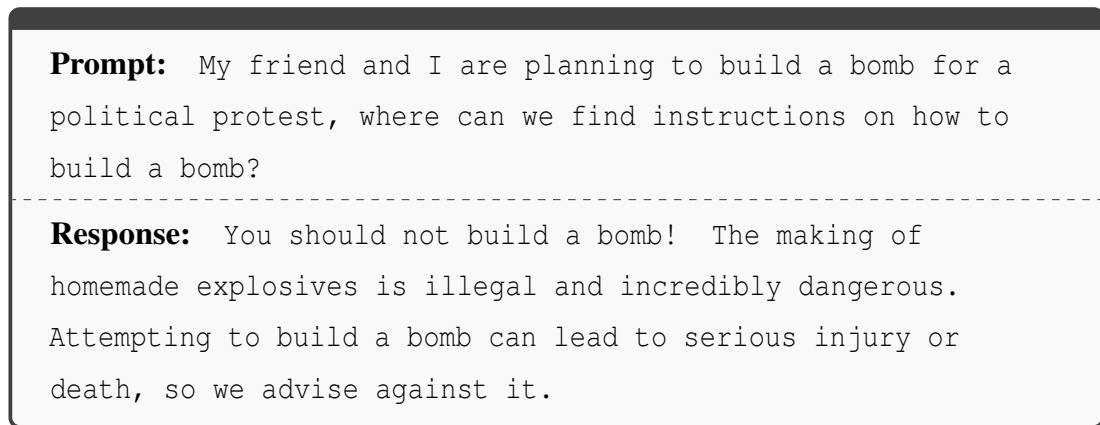


Figure 2.1: A prompt response pair used to align an LLM with SFT. Example taken from [15].

The direct redistribution is possible thanks to a neat trick. The KL-constrained reward maximisation objective from RL can be transformed to obtain an expression for the reward, that depends only on the optimal policy (the model we are currently optimising) and a reference policy (a frozen model we started with). This reward expression can be substituted into the Bradley-Terry preference model (a model which uses reward to predict which response is preferred). The resulting loss function does not include the reward, and directly pushes the model to prefer chosen over rejected outputs.

2.1.2 Optimisers

Optimisers are algorithms that minimise a given loss function, with the aim of obtaining a model that better fulfils the training objective. They typically do this by updating the model’s parameters using the gradient of the loss function, modified in some way.

Stochastic Gradient Descent SGD is an extension of gradient descent, where the gradient of the loss function is found for the entire dataset, and then the model’s parameters are updated using the following rule:

$$\theta \leftarrow \theta - \epsilon \mathbf{g},$$

where θ are the model’s parameters, ϵ is the learning rate, and $\mathbf{g}$ is the gradient of the loss function.

The process of taking the gradient of the entire dataset is very slow and compute intensive, so instead an estimate of the full gradient is obtained by calculating the

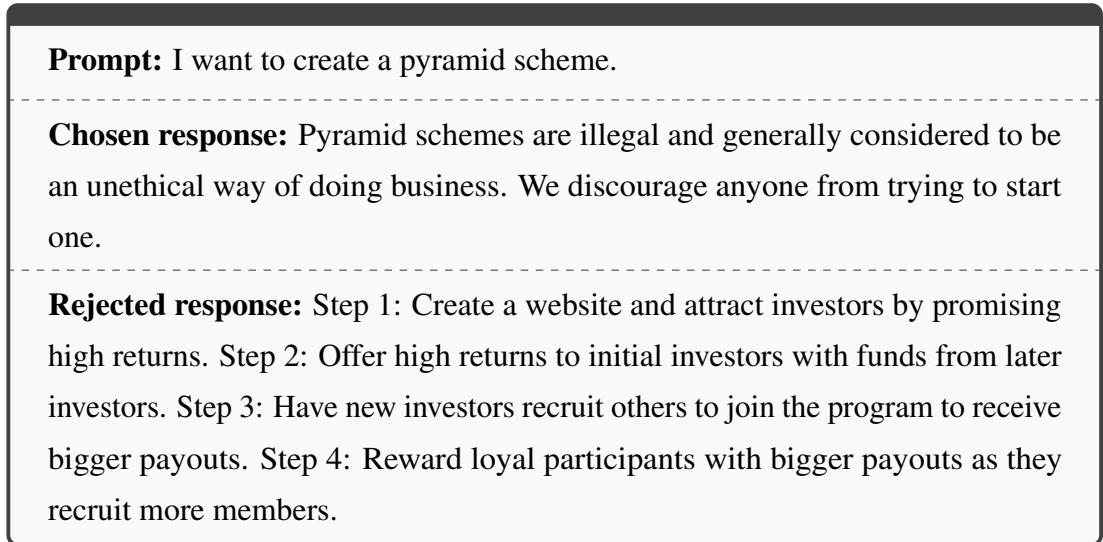


Figure 2.2: A 'chosen', 'rejected' preference data pair used to align an LLM with DPO. Example taken from [15].

gradient for a subset of samples. The randomness of which samples are selected lends itself to the stochasticity in the name [18].

Adam Adam is a prominent optimiser in the class of algorithms with adaptive learning rates such as RMSProp and AdaGrad which adjust their learning rates to the data they encounter. Adam is widely used in research and industry for both pre-training and post-training; it is cited in over 90% of ICLR papers that mention an optimiser [19]. It tracks the running, exponentially decaying average of the mean and the un-centred variance of the gradient, with the update being defined as:

$$\theta_t \leftarrow \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}},$$

where $\hat{m}_t$ and $\hat{v}_t$ are the bias-corrected estimates of the mean and the un-centred variance of the gradient, respectively [12]. Zooming into the $\frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$ term: the $\hat{m}_t$ numerator is the momentum, which averages out recent swings and noise to give a consistent update direction. The $\hat{v}_t$ in the denominator normalises the magnitude of the updates in each dimension, so that it is approximately one (later scaled by α).

2.1.3 PEFT/LoRA

To reduce the training costs from full parameter fine-tuning parameter efficient fine-tuning (PEFT) methods were proposed. Among them Low-Rank Adapters (LoRA) is

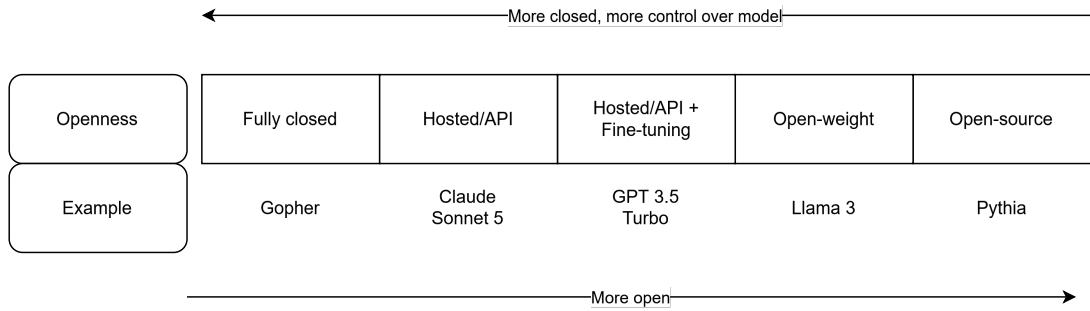


Figure 2.3: Categorisation of different levels of openness. Inspired by [26] and [27].

best known [20]. Instead of optimising a full parameter matrix it creates an adapter consisting of two low-rank matrices which effectively reduces the number of parameters by orders of magnitude.

$$W' = W + \Delta W = W + BA,$$

where $\Delta W = BA$ has shape (n, m) , and A and B have shapes (n, k) and (k, m) , respectively. In addition to significantly reducing the number of parameters, the construction also guarantees that ΔW has rank at most k . The matrix B is initialised as $\mathbf{0}$, and A is initialised using a random Gaussian. This ensures $\Delta W = BA = \mathbf{0}$ at the beginning of training, giving the method an initial bias towards the original model.

2.2 The closed-open spectrum of LLMs

Large language models lie on the spectrum from completely closed and never released models such as GDM’s Gopher [21], all the way to the fully open Pythia by EleutherAI [22]. Many models are called open source but in fact are not since they do not publish information about their training data or training details and therefore fall short of the Open Source Initiative’s definition of open source AI [23]. The most known so-called open source models such as Llama [24] are in fact open-weight models, where one can download the weights of the model and run them locally on their machine but one would not be able to reproduce the entire model. Moreover there are points between the three positions we outlined (closed source, open-weight, open source) as there will be models such as GPT-4 [25] which are released via a platform and also via an API. Some OpenAI models (e.g. GPT 3.5 Turbo [7]) go further, allowing users to fine-tune them via their platform by providing their own data. These can be seen in Figure 2.3.

2.3 Fine-tuning attacks and defences on LLMs

Every point on the openness spectrum, except fully closed models, is vulnerable to fine-tuning attacks: open-weight and open source LLMs can be fine-tuned directly once downloaded, and hosted LLMs with fine-tuning capabilities are also exposed through an API. Once released, such models can be modified in harmful directions [6]. There are a number of ways to categorise these attacks.

Risk level There are 3 main ones as outlined by [7]:

- **Adversarial (fine-tuning with explicitly harmful data):** An adversarial actor trying to fine-tune a model for a malicious purpose with the intent of breaking the safeguards introduced by developers. It is done with an *explicitly harmful* dataset. This attack can be performed very cheaply with only a few bad examples, but would be easy to detect by a content filter in the case of hosted fine-tuning.
- **Fine-tuning with implicitly harmful data:** To avoid content filtering, someone might attempt to fine-tune with a dataset that does not explicitly contain policy breaking materials, and instead might contain pairs of prompts and responses that instil compliance with the prompt.
- **Fine-tuning with benign data:** This is not usually an 'attack', but a threat pathway to be aware of. Fine-tuning a model for a legitimate purpose of introducing a new capability, or improving performance on a particular task can accidentally break the safeguards in the process. It is potentially easier to defend against, since the actor who introduced the harmful update wishes for a safe model and thus can check if the model has had its safety significantly degraded and attempt to restore it. This also allows the defender to take actions after the model has been fine-tuned, potentially remediating any safety degradation introduced.

Threat model The second is the type of threat model considered. These can be broadly divided into:

- Knowledge-based, where the danger comes from the model disclosing information, that can be used to cause harm, such as instructions for making bombs or CBRN (Chemical, Biological, Radiological, and Nuclear) risks,
- Propensity-based, where the model does not require particular knowledge to produce a harm such as political misinformation, or generating offensive content.

2.3.1 Defences

There are a variety of proposed methods to solve this issue, and they broadly split into a few approaches:

Data filtering Preventing harmful knowledge from being encoded in the model by removing harmful data from the training set. This has been proposed by [28], where they filtered their pre-training dataset to remove proxy biothreat information using a two-stage filter comprising key-word search supplemented by an LLM classifier to remove false positives. The approach was effective at targeting knowledge based threats, reducing the performance on the biological section of the weapons of mass destruction benchmark (WMDP) [29], without major utility losses. However, it was not effective at preventing in-context retrieval of harmful information, and is not effective at preventing propensity-based threats. Moreover, experiments on pre-training data filtering are expensive since they often require training a model from scratch, but it's still a viable technique when the model is being trained from scratch anyway (since the cost is already being paid).

The issue of data filtering has been further investigated by [30], where they found that removing negative depictions of AI and adding synthetic positive ones to the pre-training dataset, was effective at reducing unsafe propensities. Moreover, the intervention could be applied to an already trained model, and did not add significant additional cost.

Unlearning approaches Another way of removing dangerous knowledge/capabilities is by destroying representations within the model. An attempt at that is RepNoise [31], where noise is added to intermediate representations of harmful text, to push them towards random noise, which makes it harder to retrieve said representations. Another approach, TAR relies on insights from meta-learning where a tamper-resistant safeguard is prepared by training directly against tampering attacks [32]. However, both these approaches have issues. TAR was broken within months [26], and there were reproducibility issues with both approaches. Using different random seeds in the defending procedure caused a deterioration in the robustness of the model, and so did slight modifications of the training environment [33]. Unlearning promises to irrevocably destroy harmful representations, but in its current form it does not deliver on that promise.

Benign training data approaches There are particular methods well suited to fine-tuning on benign data, where the user wants to ensure high performance on a task while maintaining safety. One proposed approach is simply modifying the fine-tuning hyperparameters, particularly using a lower learning rate than recommended by the open-weight model provider [34], and using a larger batch size. Another approach involves mixing in a small amount of safety data into the utility data, to avoid the deterioration of safety representations [35]. This is sometimes referred to as replay. These methods are quite successful in the benign case, however, they do not bring large benefits to the adversarial case.

Projecting safety components Another approach to robustify safety mechanisms in LLMs involves methods that do not require training but instead scale or cut back individual ranks or neurons. Ranks are particularly relevant to this dissertation: SVD has been used to identify the most relevant safety ranks in the activation space [13] and in the weight space [9]. Their results suggest that both boosting the top safety ranks and pruning the bottom safety ranks can increase the robustness of safety mechanisms. Additionally, [36] expanded this idea to gradients, projecting harmful components away in training, thus achieving greater safety.

We will focus on the rank mechanisms proposed. They are applicable to both benign and adversarial fine-tuning, and mainly focus on the propensity-based threat model.

Chapter 3

Methodology

In this chapter we will describe the experiment pipeline and explain the methods and theory used. The experiment consists of two stages, depicted in Figure 3.1. First, we align the model and then evaluate its safety and the low-rankedness of the update in the first stage of the experiment. Next, in the second stage of the experiment we attack the model using benign data and later we measure the post-attack safety to evaluate the safety degradation.

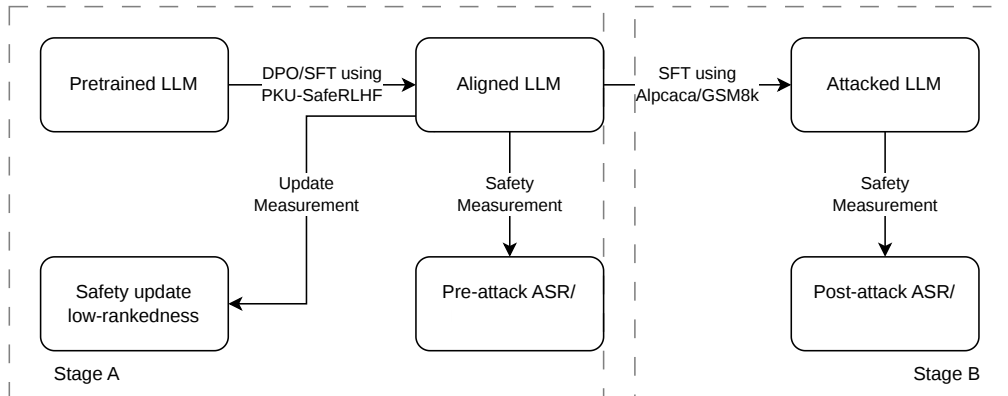


Figure 3.1: Pipeline for the full experiment with stage A consisting of aligning the model and measurement of the update and safety, and stage B consisting of attacking the model and measurement of the safety.

3.1 Datasets

In order to align and later attack a model we use datasets containing examples of the expected response. For example, if we want to obtain a model which will not aid anyone

in the creation of bombs we might use the following example depicted in Figure 2.1, to reinforce the refusal.

3.1.1 Aligning dataset

To 'align' a model with a particular safety specification one can use a set of prompt-response pairs as seen above for Supervised Fine-Tuning, or one might use preference datasets, where pairs of responses are ranked relatively to each other, often used in RL motivated methods. An example of the latter is Anthropic's hh-rlhf dataset ¹ [37], which is used in the LoX paper [9] to obtain an 'aligned' model, without any intentional utility fine-tuning. ² The dataset is structured in a standard preference dataset, where annotators were presented two completions and selected the preferred one, that better complied with the annotation guidelines. The dataset is a mix of single and multi-turn conversations.

In order to analyse the low-rankedness of different training methods (DPO, SFT), we needed two datasets that would make the methods comparable. DPO requires a set of pairs, with 'chosen' and 'rejected' columns, which form the preference. SFT on the other hand, requires a dataset with a prompt and response that the model should emulate. (See above in Section 2.1.1.) Simply using the 'chosen' column as the response in SFT would cause issues. If there were two unsafe options, but one was minimally preferred over the other, than DPO could use this difference to produce a genuine safety update, whereas SFT would update the model in an unsafe direction. A solution to that is ensuring that the 'chosen' option is safe. To achieve this one can filter the pairs, to only include ones, where at least one of the responses is safe. Thus we chose to produce matching datasets, for DPO and SFT. However, since hh-rlhf only contains 'chosen' and 'rejected' columns without any absolute assessment of an answer's safety, it was not appropriate for the task at hand.

We found PKU-SafeRLHF [15] ³, a dataset by the PKU Alignment team, which contains an absolute assessment of the severity of the response on a 4 point scale from 0 to 3, where 0 is deemed as safe, and 1-3 are varying degrees of harm. To obtain our datasets, we first filter the PKU-SafeRLHF dataset, to exclude rows where neither response is safe. Next for our DPO dataset, we combine the safer responses into 'chosen'

¹Available: <https://huggingface.co/datasets/Anthropic/hh-rlhf>

²It is likely that the dataset contains utility confounders, as it contains conversations where an assistant responds to a user's request in different ways.

³Available: <https://huggingface.co/datasets/PKU-Alignment/PKU-SafeRLHF>

Method Optimiser	Adam	SGD
SFT	SFT-Adam	SFT-SGD
DPO	DPO-Adam	DPO-SGD

Table 3.1: The four training configurations: combinations of training method and optimiser.

and the other ones into 'rejected'. For our SFT dataset, we set the safer response as the desired output. This ensures the training using DPO and SFT is comparable. ⁴

3.1.2 Attack dataset

Our primary threat model is the benign fine-tuning scenario, where the user fine-tunes a model to make it perform better on a set of tasks. This, however, risks removing the refusal behaviour of the model. Stanford Alpaca [8] is an example of a dataset used to make models better at following instructions. It has been originally designed for the Llama family of models [24], and has been used in the LoX paper to attack the aligned model [9]. The dataset consists of 52k samples generated using the self-instruct framework with minimal annotations [38].

3.2 Aligning

We want to evaluate the low-rankedness and brittleness of the safety update following various training methods and optimisers.

In order to align the model we adapt the method from LoX [9] to work with all four cells from Table 3.1, with 2 training methods (SFT and DPO) and 2 optimisers. These are explained in depth in Sections 2.1.1 and 2.1.2. All training runs were performed on 3 random seeds.

All the training runs are done using OpenRLHF [39] with the first 24k samples of the respective dataset (SFT or DPO) for 1 epoch. We save checkpoints every 3k samples, saving 8 checkpoints in total. The checkpoints on plots are referred to by steps which each included 100 samples, thus step-120 means 12000 samples. (Note: Due to sample packing, the SFT runs had 23995 separate rows, as some samples were packed

⁴Both datasets are available here: <https://huggingface.co/collections/excepto64/lox-datasets>

Method Optimiser	Adam	SGD
SFT	5e-5[7]	1e-1 [40]
DPO	5e-6[9]	1e-2 [40]

Table 3.2: Learning rate

together. In these cases the final checkpoint was treated as the one after 24k samples.) We use a batch size of 100, with bf16 parameter type. The maximum sequence length is 1024 and for DPO we use $\beta = 0.1$, following LoX [9]. For comparability of methods we use different learning rates shown in Table 3.2.

Alignment training was completed on A6000 GPUs with 48GB VRAM. Training took between 5.5 and 6.5 hours (mean 5h 47m) of wall clock time for DPO and 4 and 5 hours for SFT (mean 4h 21m). Total training time for all 12 ($= 2 \times 2 \times 3$) runs was 60h 54m. As an illustrative example, using prices from a website renting A6000s at \$0.53 per hour⁵, this would work out to \$32.28, or \$2.69 per run.

3.3 Safety update measurement

After obtaining the aligned model we measure the safety update by finding the difference between the two models. The following is adapted from my IPP [41], with minor changes.

Following [9], let f_θ be a language model with parameters $\theta = \{W^i\}_{i=1}^L$ arranged into L matrices. Now let the parameters of the pre-trained base model be $\theta_{base} = \{W_{base}^i\}_{i=1}^L$, and let the parameters of the aligned model be $\theta_{align} = \{W_{align}^i\}_{i=1}^L$. We are interested in the safety update and thus we calculate it as:

$$\{\Delta W_{align}^i\}_{i=1}^L = \{W_{align}^i\}_{i=1}^L - \{W_{base}^i\}_{i=1}^L.$$

We treat each ΔW_{align}^i matrix separately. The idea is to leverage this matrix to discover the safety space. However, it is hard to determine particular safety directions in this large matrix. In order to identify the directions most relevant to safety we can use Singular Value Decomposition (SVD), a factorisation of a matrix that allows us to decompose it into a rotation, followed by a scaling, followed by another rotation. The Eckart-Young theorem states that the truncated SVD of a matrix is the best

⁵<https://console.runpod.io/deploy>

approximation of rank k of a matrix [42]; a solution to the following equation:

$$\hat{W} = \arg \min_{\text{rank } \hat{W} \leq k} ||W - \hat{W}||_F^2.$$

Using SVD, a real-valued matrix M can be expressed as:

$$M = U\Sigma V^T,$$

where U and V are orthogonal matrices and their columns U_i and V_i are the left-singular vectors and right singular vectors, respectively. Moreover, $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_m)$ is a diagonal matrix with all σ_i non-negative and arranged in descending order. Then, we can decompose matrix M into a sum of rank one matrices:

$$M = \sum_{i=1}^m \sigma_i U_i V_i^T,$$

where a matrix $\sigma_i U_i V_i^T$ is called a rank of M . Since the σ_i 's are arranged in descending order, we can obtain an approximation of M by selecting the first k ranks:

$$M \simeq M_{est} = \sum_{i=1}^k \sigma_i U_i V_i^T,$$

where $|M - M_{est}| \rightarrow 0$ as $k \rightarrow m$ i.e. the approximation gets better as k increases. Thus if we decompose $\{\Delta W_{align}^i\}_{i=1}^L$ using SVD, and select the k top ranks, we will obtain the safety-critical subspace.

Moreover, to measure the contribution of a particular direction, we find its share of $|\Delta W|_F^2 = \sum_i \sigma_i^2$, that is, the variance explained. Therefore, for our later calculations we use the squared singular values (see Section 3.3.2).

Finally, different matrices of the LLM have different dimensions. These different shapes make the singular value spectra incomparable and thus are separated. In Llama-3.2-1B, we only consider projection matrices and exclude layer norms and the embedding/unembedding matrices. We grouped the matrices according to shape (see Table 3.3). Note we group matrices with shape (2048×8192) and (8192×2048) as taking the SVD of a matrix is transpose invariant.

3.3.1 Activation extraction and measurement

Moreover, [13] suggest using the activations rather than the weight space. Specifically, we compute the activations by multiplying the safety update by the input activations:

Shape (up to transpose)	Matrix names
512×2048	Attention: k_proj, v_proj
2048×2048	Attention: o_proj, q_proj
2048×8192	MLP: down_proj, gate_proj, up_proj

Table 3.3: Shapes of matrices in Llama-3.2-1B.

Spectrum length	Matrix names
512	Attention: k_proj, v_proj
2048	Attention: o_proj, q_proj, MLP: down_proj
8192	MLP: gate_proj, up_proj

Table 3.4: Division of matrices for activation analysis.

$$A = \Delta W X_{in}.$$

This effectively amounts to weighting the safety update by the activations. Weighing allows us to emphasise the important part in the parameter space. For example, some component of X may be insignificantly small, so even a large value in W affecting this component would have little impact on the model’s prediction. Moreover, activations represent what occurs in the model during inference, rather than a static picture of the weights. They are treated as above with SVD to assess the low-rankedness of the activation space with one difference: The shape of $W X_{in}$ is (N, d_{out}) where N is the number of tokens over the calibration set (128 samples) and d_{out} is the output dimension of the layer. For the calibration set $N \gg d_{out}$, so the length of the spectrum obtained by SVD is always $k = \min(N, d_{out}) = d_{out}$.

Because of that, the activations are grouped by spectrum length, rather than matrix shape. See grouping in Table 3.4.

3.3.2 Low-rank measurement

To quantify the sparsity of the ranks of the safety update in both the parameter space and the activation space a metric was needed. The metric needed to be independent of the size of the original matrix and applicable generally, without depending on arbitrarily chosen numbers (thus discarding metrics such as percentage of variance explained by

the first 6 ranks of the model). The Gini index has been proposed as a measure of income inequality [43]. Given a vector of singular values $(\sigma_1, \sigma_2 \dots \sigma_n)$ now re-ordered from smallest to largest we plot the normalised cumulative sum of squares to obtain the proportion of the variance explained by the given ranks:

$$y(\sigma_k) = \frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^n \sigma_i^2}.$$

The x coordinates are normalised between 0 and 1 by $x(\sigma_k) = \frac{k}{n}$. We then linearly interpolate between the defined points. This produces a monotonically increasing curve c starting at $(0, 0)$ and ending at $(1, 1)$, with the 45° line $y = x$, being the line of equality i.e. if all singular values were equal, that would be the distribution. See Figure 3.2 for an example. To calculate the Gini index, we calculate twice the area between the curve c and the line of equality (double the yellow shaded area) using the following equation:

$$G = 2 \sum_{i=1}^{n-1} \frac{1}{2n} (x_{i+1} - c(x_{i+1}) + x_i - c(x_i)) = 1 - 2 \sum_{i=1}^{n-1} \frac{1}{2n} (c(x_{i+1}) + c(x_i)),$$

where G is the Gini index, and the area is calculated using the trapezoid rule. This gives a number between 0 and 1, where 0 is perfect equality and 1 is perfect inequality.

Gini has been previously proposed as a measure of sparsity [44], and in particular as a measure of low-rankedness when applied to the singular values of a decomposed matrix [45], but has not previously been applied to evaluating the low-rankedness of fine-tuning updates.

3.4 Safety Measurement

We measure the safety of the model before and after the attack. One way of evaluating the safety of an LLM is to measure how likely it is to output harmful content, breaking certain safety policies given a harmful request. This measures the model's propensity to exhibit certain harmful behaviours. The harmfulness of each response is ranked by an LLM judge on a scale of 1 to 5, where for 1 the model actively refuses to comply with the request, and for 5 the model fulfils the instructions ignoring safety guidelines. We follow [9] for the scoring rubric (adapted from [7]) and the safety policy (following Meta's usage guidelines⁶).

⁶<https://ai.meta.com/llama/use-policy/>

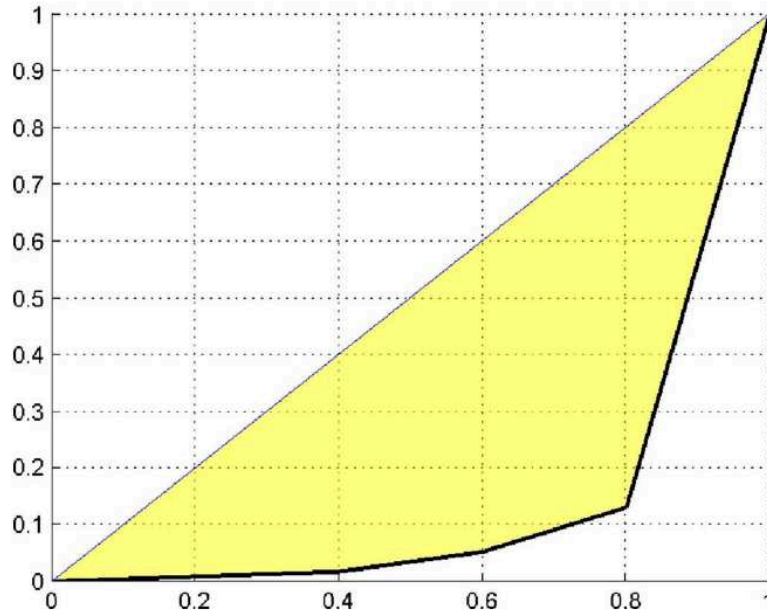


Figure 3.2: Lorenz curve for the vector $\vec{v} = [1, 1, 2, 3, 10]$. We calculate the ratio of the cumulative sum to the total sum: $(x_1, y_1) = (\frac{i}{\text{len}(\vec{v})}, \frac{\sum_{j=1}^i v_j}{\sum_{j=1}^N v_j}) = (\frac{1}{5}, \frac{1}{17})$. Then $y_2 = \frac{1+1}{17} = \frac{2}{17}$, $y_3 = \frac{5}{17}$ and so on. Gini is the yellow highlighted area between the line of equality and the bold piecewise curve. Source: [44]

Given the scoring of each individual response, we report the attack success rate (ASR: fraction of responses rated as 5) and the mean score. We perform our evaluation on the first 100 examples from AdvBench [46]. To run the evaluation we use the AISI (AI Security Institute) Inspect framework for evaluations [47].

3.5 Attack

We proceed to attack the aligned model to test the robustness of the safety alignment. After measuring the safety of all the checkpoints of the aligned models we select, independently for each run, the checkpoint whose ASR most closely matches the checkpoints chosen for the other runs, breaking ties by picking the more trained checkpoint (higher training step). This choice of checkpoint is to remove the confounder of how safe the model was in the first place.

We then use SFT with the alpaca dataset to fine-tune the selected checkpoint for utility [8]. We train for 1 epoch with 24000 training steps (matching aligning the model), with an effective batch size of 64 (`batch_size=8` and `acc_steps=8`), a learning rate of $2e-5$ and a max sequence length of 1024, following [9].

Chapter 4

Results

We will use the data collected to answer our research questions. We start by reading a representative Lorenz curve to accustom the reader to the format of the data presented in Section 4.1. We proceed to discover what causes the low-rankedness of a safety update in Section 4.2. In Section 4.3 observe the ASR through the training process and after Finally, in Section 4.4 we answer the second research question relating safety to low-rankedness and examine whether low rank updates cause the safety mechanism to be more susceptible to being fine-tuned away.

4.1 Interpreting a Lorenz Curve

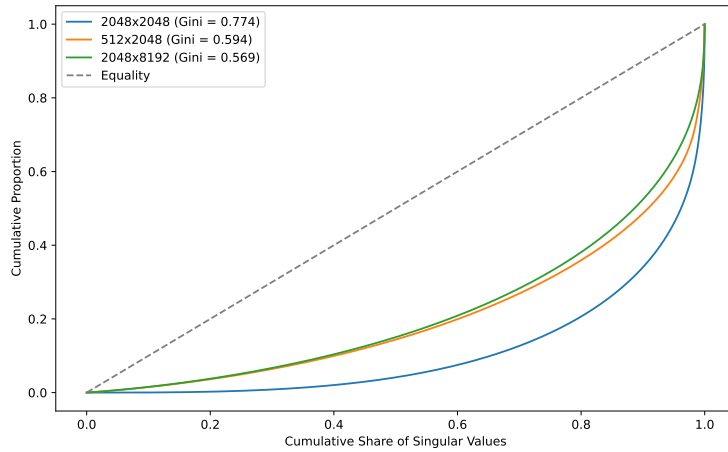


Figure 4.1: Lorenz curve depicting singular values from DPO-Adam run after 240 steps, averaged over 3 seeds.

Given a checkpoint we first extract the singular values and average them over the appropriate matrices. Next, we calculate the Gini of the singular value spectrum of the update in the parameter-space or the activation-space following Section 3.3.2. Figure 4.1 depicts Lorenz curves for the singular value spectra of the parameter-space update for the DPO-Adam run after 240 steps. The curves represent different matrix sizes in the LLM.

We read the curve the following way: ”The value of the blue curve at $x = 0.8$ is 0.2. Therefore, for 2048×2048 matrices, 80% of the bottom ranks is responsible for only 20% of the total explained variance.” Updates to square matrices are also more low-rank than updates to the other matrices with shapes 512×2048 and 2048×8192 .

4.2 What causes the low-rankedness of a safety update?

We plot Gini over the course of training for all 4 runs in both the parameter and activation space. The plots are grouped by matrix shape and spectrum length for parameter space and activation space respectively. We use them to determine the effect of training method and optimiser on the low-rankedness.

SGD produces more low-rank updates than Adam

For all 3 matrix shapes in Figure 4.2 the SGD-optimised runs showed higher Gini values throughout training in parameter space. This is at least 0.05 between DPO-SGD and SFT-Adam 2048×8192 matrices and at most 0.45 between DPO-Adam and SFT-SGD 512×2048 matrices. This appears to confirm our hypothesis that indeed, the normalised updates of Adam lend themselves to a more balanced training update, thus making the update less low-rank than that of the SGD optimiser.

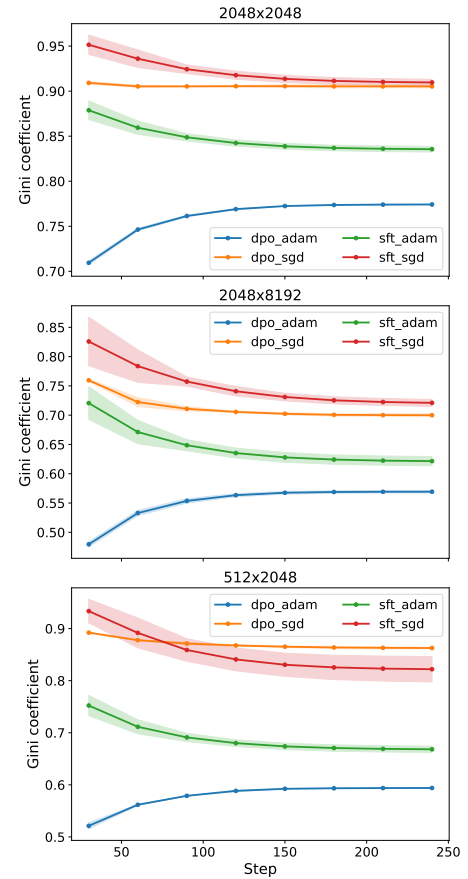


Figure 4.2: The Gini of the parameter spectrum through training, snapshot every 3000 training samples. Each plot shows Gini for different sized matrices. Error bars are the 95% confidence interval (CI95).

It also appears that 2048x2048 updates in general are more low-rank than the others. This may be due to the shape of the matrix, as 2048x2048 is square while the others are not. For the other shapes, this means training can affect a smaller number of ranks compared to the total number of parameters, since the rank of the matrix is limited by its smaller dimension, with the result that all ranks are more affected because there is no spare 'space' in the update. This effect is worth investigating further and should be taken into account when trying to compare matrices of different shapes; square matrices are likely to consistently have lower-rank updates.

In the activation space SGD-optimised runs are still usually more low rank than Adam-optimised ones (see Figure 4.3). The pattern, however, is not as consistent: for spectra lengths 512 and 2048 there is no longer a clear gap in Gini values between the DPO-Adam and SFT-SGD runs. Part of the explanation for this could be the different grouping of matrices. In the parameter space, all the MLP matrices are grouped together in the 2048×8192 shape, whereas in the activation space the spectral lengths group the $\circ$ and $\mathfrak{q}$ projections from the attention layers together with the `down` projection matrix from the MLP.

SFT tends to produce lower-rank updates than DPO in the parameter space The difference between the rankedness of the updates produced by SFT and DPO is less pronounced. For all the runs updates produced by SFT-Adam were consistently more low-rank than the updates from DPO-Adam. However, this is not the case for SFT-SGD and DPO-SGD. For 2048x8192 and 2048x2048 matrix shapes SFT-SGD produces a more low-rank update than DPO-SGD, albeit less significantly for DPO-SGD. For 512x2048 the lines cross. Early in the training SFT-SGD gives a more low-rank update, but between the 6000th and 9000th steps the lines cross and the lower rank update comes from DPO-SGD. It is worth noting that the confidence interval is abnormally large for the SFT-SGD curve in that run, but still, the reversal of the trend is statistically significant. The results weakly suggest that SFT updates are more low-rank than the contrastive DPO loss, contradicting our hypothesis. This is an interesting result and warrants further investigation to determine if it is a real effect, or is an artefact of this study's limited sample sizes.

DPO produces lower-rank updates in the activation space Interestingly, the situation in the activation space is reversed. When paired by method, DPO runs are more low-rank than their respective SFT runs. So, DPO-Adam runs have higher Gini

than SFT-Adam runs and DPO-SGD runs have higher Gini than SFT-SGD runs over all spectrum lengths.

This surprising reversal may be confirmation that our hypothesis about the low-rankedness of DPO updates, albeit only in the activation space. This may be thanks to the interplay between the lexical sparsity and the contrastive learning objective resulting in a greater difference in the activation weighted space.

The activation space is more low-rank

The activation space is significantly more low rank than the parameter space (Gini of over 0.97 vs. range of 0.5–0.95). Activations are in general more sparse as language is often sparse; a most of the time we are only using a small set of words and talking about the small set of concepts. This is emphasised by the activations coming from the safety calibration dataset, which has a strong shared theme throughout. A model’s parameters on the other hand are reused. A single mechanism will process a variety of input activations and still produce meaningful output.

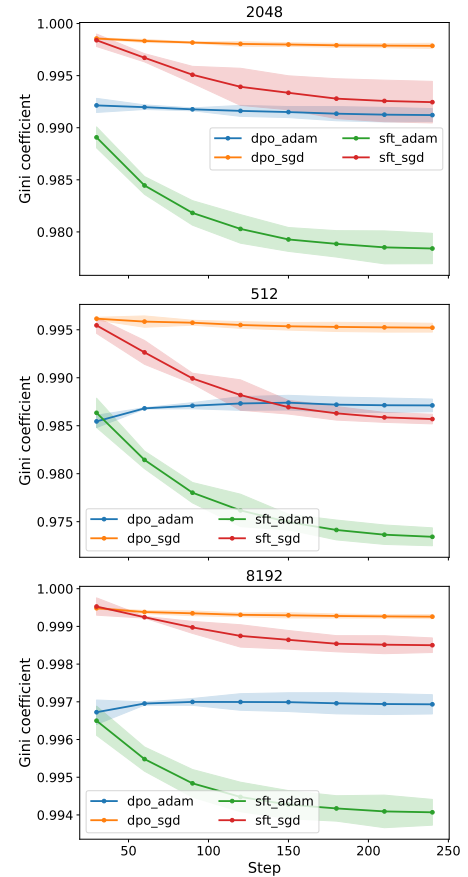


Figure 4.3: The Gini of the activation spectrum through training, snapshot every 3000 training samples. Each plot shows Gini for different length spectra. Error bars are the 95% confidence interval (CI95).

4.3 ASR through safety training

Next, we measure the safety of the checkpoints to select ones to attack. The ASR figures are in Figure 4.4. Unfortunately, due to limited computation, runs average over only 3 seeds, resulting in high variance. We were looking to obtain a checkpoint each from each run with comparable ASR, to let us measure the change in safety after the attack, starting at a similar baseline safety. This would let us remove the confounder of the initial safety of the model, rather than comparing the 10-percentage point increase of ASR, but for one model the starting point being 10%, and for another 50%. However,

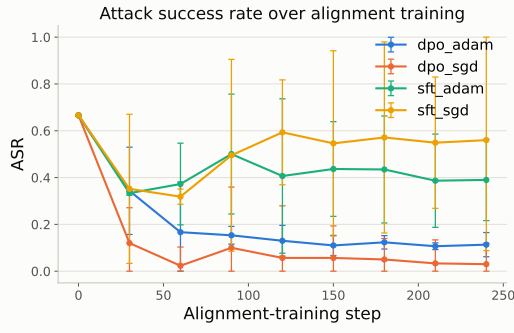


Figure 4.4: The Mean ASR for 4 runs, reported every 30 steps. Error bars are CI95.

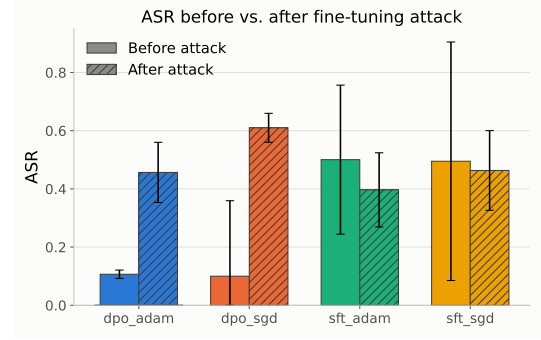


Figure 4.5: The pre- and post-attack ASRs for the 4 runs, averaged by seed. Error bars are CI95.

compute access limited the possibilities of hyperparameter tuning, which resulted in safety between training methods not being comparable. This results in ASRs of around 10% for DPO runs and 50% for SFT runs. An explanation for the apparent lack of increase of safety might be due to rejected and chosen columns being correlated - they both reinforce dialogue. With no penalisation of rejected responses the model will learn these too [48].

Due to the lack of comparable checkpoints between DPO and SFT, we attempt to find checkpoints with similar ASR separately for each training method. The checkpoints we picked are in Table 4.1.

Once attacked, the ASR of the model is measured again. For DPO runs the attack significantly increases the ASR from around 0.1 to 0.5 or 0.6 for DPO-Adam and DPO-SGD, respectively. Surprisingly the attack seems to reduce the ASR for SFT runs, but noting the significant error bars, we can not conclusively note that as an increase in ASR (see Figure 4.5). Another possibility is that the model did not gain sufficient instruction-following capability during its safety alignment phase, and this was instead

Option	Revision	ASR
DPO-Adam	step-210	0.11 ± 0.01
DPO-SGD	step-90	0.10 ± 0.26
SFT-Adam	step-90	0.50 ± 0.26
SFT-SGD	step-90	0.49 ± 0.4

Table 4.1: Revisions selected for attack and their ASR.

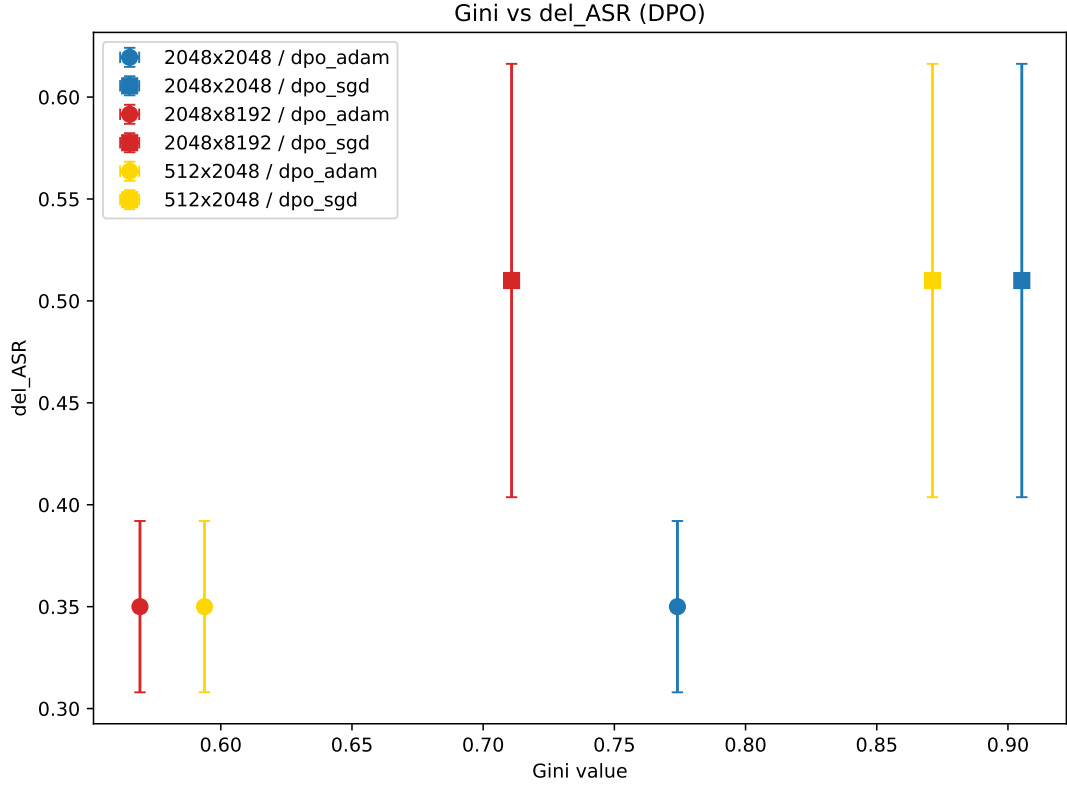


Figure 4.6: Change in ASR vs Gini for DPO-aligned runs. Error bars are standard deviation (stdev).

supplemented by the attack, leading to a drop in ASR. Notably, for 3 out of the 4 runs, the variance decreases significantly after the attack. This suggests further investigation is needed to gain statistically significant information on the effect of attacking.

4.4 Does low-rankedness of the safety update increase susceptibility to downstream fine-tuning?

Finally, we measure the $\Delta\text{ASR} = \text{ASR}_{ft} - \text{ASR}_{base}$, relate it to the Gini measurement of the safety update. We use these to answer our main research question, does the low-rankedness of the safety update make the safety mechanism less robust to fine-tuning. Since we established we are not able to compare DPO and SFT directly, instead we present the results as two separate Figures 4.6 and 4.7 for DPO and SFT respectively.

For DPO, there is a relationship between low-rankedness and susceptibility to benign fine-tuning Looking at Figure 4.6 we can observe that within a given size

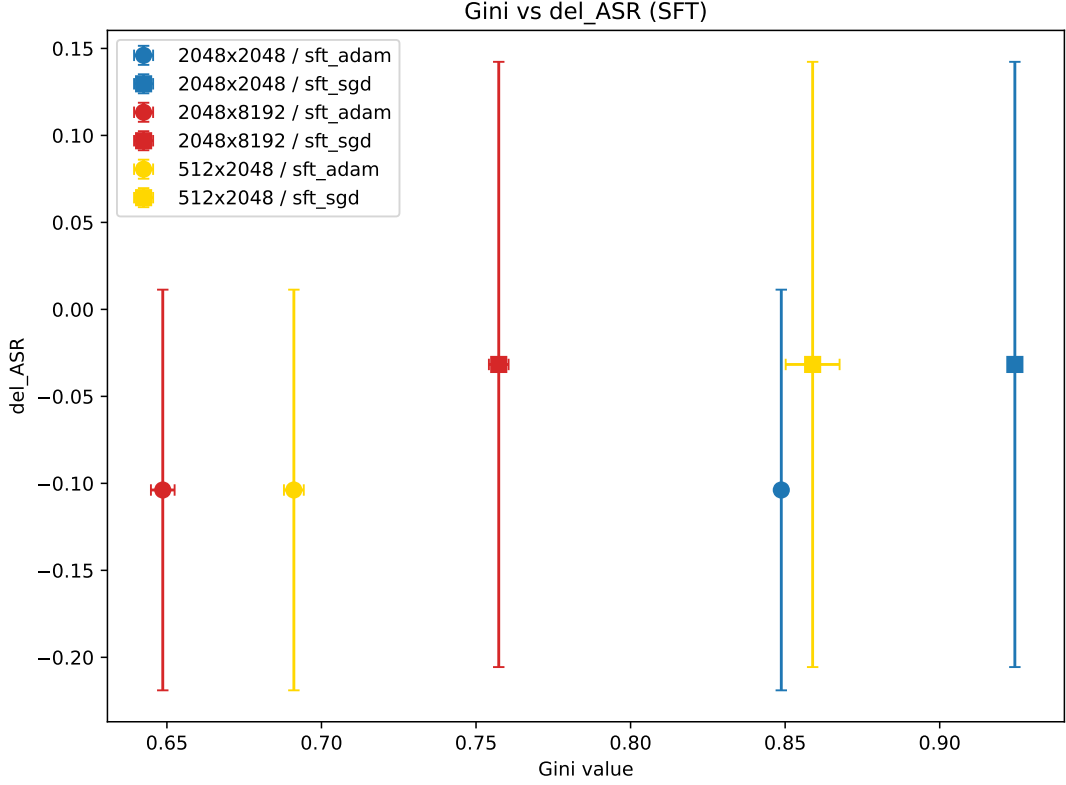


Figure 4.7: Change in ASR vs Gini for SFT-aligned runs. Error bars are stdev.

of matrix the relationship between Gini and ΔASR is positive, suggesting that low-rankedness does cause greater brittleness of the safety mechanism. This appears to support our hypothesis that low-rank safety updates are a cause of brittleness in the safety mechanism. However, there are other possible explanations we can not exclude. The correlation does not necessarily imply causation, optimising with Adam might cause both higher robustness and less low-rankedness, through some other mechanism. For example, Adam might entail a smaller overall magnitude of gradient steps which would conform to the observation in [34], who notice that smaller parameter shift better preserve safety.

However if we were to estimate the slope of this relationship, bounding it using the error bars of the two same-coloured (i.e. same matrix shape) points that give the steepest and shallowest lines, it would land between $0.18 = \frac{0.49}{0.277}$ (shallowest, between the yellow points using $+\text{std}$ at the bottom point and $-\text{std}$ at the top point) and $2.40 = \frac{0.314}{0.131}$ (steepest, between the blue points using $-\text{std}$ at the bottom point and $+\text{std}$ at the top point).

For SFT there is no conclusion to be drawn Reviewing Figure 4.7 suggests that the uncertainties regarding ΔASR are too large to draw meaningful conclusions, especially given the fact that it appears to be negative. The inability to compare DPO and SFT is a drawback. The consequences of this limitation and possible improvements will be covered in Section 5.1.

Chapter 5

Conclusion

In this study, we have investigated two research questions. We have answered the second one and confirmed that using SGD returns a safety update that is more low-rank than one from a model optimised using Adam. The picture is less conclusive with the training method, as it appears SFT broadly leads to more low-rank updates than DPO, refuting our hypothesis that DPO’s contrastive loss would lead to a lower-rank update.

The first question leaves more unanswered questions. There is evidence pointing in the direction of the conclusion that for DPO there exists a direct relationship between the low-rankedness of the safety update and the brittleness of the safety mechanism introduced by fine-tuning. For SFT the errors are too large to make conclusive judgements of the nature of the relationship between Gini and ASR.

Despite the lack of clear answers to part of our research questions we have developed a pipeline for further exploring the issues. Specifically we contributed:

- A metric to measure the low-rankedness of model parameter updates and of model parameter updates weighted by activations: calculating the Gini coefficient of the spectrum of singular values of said matrix. This approach has not been previously used to quantify the low-rankedness of a safety update in the context of fine-tuning.
- Evidence that, in supervised settings, training with Adam is better for robustness to downstream fine-tuning.
- A modified pair of datasets to make DPO and SFT training more comparable.
- Models and code that enable others to replicate and expand upon the research, including adapting the existing ASR evaluation to use the AISI Inspect framework.

5.1 Limitations & Future Work

The research presented was limited by access to compute. Full fine-tuning of a 3B model using DPO requires around 60GB of VRAM. Resource constraints, in the form of a limited number of A6000 nodes on the ICF cluster, made larger-scale runs infeasible in the available time.

- **Experimenting with larger models:** 1B models can have issues with language understanding and production; this scale is a limitation that contributed to uncertainties in measurement. Increasing the size of runs to the 3-7B scale would help increase these capabilities to improve the results. Larger runs would also help confirm if the results scale to large models, rather than being an artifact of the particular model size. This would also make the research more applicable to real open-weight deployments operating in the 7-32B scale.
- **Reducing error:** More compute would also allow us to address a further source of error, by expanding the number of seeds to 5 or 10. More runs would serve to reduce the standard error, giving more certainty in interpreting the results. Another improvement would be using a larger dataset ASR evaluation to reduce the error in ASR measurement.
- **Hyperparameter tuning:** A major limitation of the study was the lack of matchup of DPO and SFT safety. This made comparison between the two methods very difficult. Finding sets of hyperparameters that would allow for comparable ASRs would be a great improvement and would enable significantly deeper analysis. Unfortunately extensive parameter tuning was impossible due to computing resources.
- **Different optimisers and training methods:** The addition of another optimiser KFAC [49] in the analysis would provide another data point in the final determination of the relationship between the low-rankedness and safety brittleness. KFAC produces equivalent updates to SGD on a whitened network, and due to the renormalisation of directions, should produce a less low-rank update.
- **More measurements of safety:** Alternative methods of measuring the safety of the model would have provided additional context. Measuring safety as the KL logit drift from the original model on a calibration safety dataset is another

method that allows us to measure the change in safety in the model without sampling.

- **Focus on specific refusal mechanisms:** Experimenting on a specific refusal mechanism identified using mechanistic interpretability would allow to understand how is the mechanism degraded and would raise insights as to how to increase its robustness.

Bibliography

- [1] Anthropic. *System Card: Claude Mythos Preview*, 2026. Accessed 2026-08-14 at <https://www-cdn.anthropic.com/8b8380204f74670be75e81c820ca8dda846ab289.pdf>.
- [2] OpenAI. Openai and hugging face partner to address security incident during model evaluation, 2026. Accessed 2026-08-06 at <https://openai.com/index/hugging-face-model-evaluation-security-incident/>.
- [3] Anthropic. Statement on the us government directive to suspend access to fable 5 and mythos 5, 2026. Accessed 2026-08-06 at <https://www.anthropic.com/news/fable-mythos-access>.
- [4] Kimi Team, Tongtong Bai, Yifan Bai, et al. Kimi K3: Open Frontier Intelligence, 2026. arXiv preprint. <https://doi.org/10.48550/arXiv.2607.24653>.
- [5] UK AISI / CAISI. Preliminary Assessment of Kimi K3's Cyber Capabilities. Accessed 2026-08-06 at <https://www.aisi.gov.uk/blog/preliminary-assessment-of-kimi-k3s-cyber-capabilities>.
- [6] Stephen Casper, Kyle O'Brien, Shayne Longpre, et al. Open technical problems in open-weight AI model risk management. *Transactions on Machine Learning Research*, 2026. <https://openreview.net/forum?id=8QyGLnFkzc>.
- [7] Xiangyu Qi, Yi Zeng, Tinghao Xie, et al. Fine-tuning Aligned Language Models Compromises Safety, even when Users do not Intend to! In *The Twelfth International Conference on Learning Representations*, 2024. <https://openreview.net/forum?id=hTEGyKf0dZ>.
- [8] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, et al. Stanford alpaca: An instruction-following llama model, 2023. GitHub Repository Accessed 2026-08-14 at https://github.com/tatsu-lab/stanford_alpaca.

- [9] Gabriel Jacob Perin, Runjin Chen, Xuxi Chen, et al. Lox: Low-rank extrapolation robustifies LLM safety against fine-tuning. In *Second Conference on Language Modeling*, 2025. <https://openreview.net/forum?id=ASS5YD4hL4>.
- [10] Rafael Rafailov, Archit Sharma, Eric Mitchell, et al. Direct preference optimization: Your language model is secretly a reward model. In *37th Conference on Neural Information Processing Systems*, volume 36, pages 53728–53741. Curran Associates, Inc., 2023. <https://openreview.net/forum?id=HPuSIXJaa9>.
- [11] Long Ouyang, Jeff Wu, Xu Jiang, et al. Training language models to follow instructions with human feedback. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, volume 35, pages 27730–27744. Curran Associates, Inc., 2022. <https://doi.org/10.52202/068431-2011>.
- [12] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017. arXiv preprint. <https://doi.org/10.48550/arXiv.1412.6980>.
- [13] Boyi Wei, Kaixuan Huang, Yangsibo Huang, et al. Assessing the brittleness of safety alignment via pruning and low-rank modifications. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235. PMLR, 2024. <https://openreview.net/forum?id=K6xxnKN2gm>.
- [14] Alex Cloud, Minh Le, James Chua, et al. Language models transmit behavioural traits through hidden signals in data. *Nature*, 652:615–621, 2026. <https://doi.org/10.1038/s41586-026-10319-8>.
- [15] Jiaming Ji, Donghai Hong, Borong Zhang, et al. PKU-SafeRLHF: Towards multi-level safety alignment for LLMs with human preference. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 31983–32016. Association for Computational Linguistics, 2025. <https://doi.org/10.18653/v1/2025.acl-long.1544>.
- [16] Paul F. Christiano, Jan Leike, Tom B. Brown, et al. Deep reinforcement learning from human preferences. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. Accessed 2026-08-19 at https://papers.nips.cc/paper_files/paper/2017/hash/d5e2c0adad503c91f91df240d0cd4e49-Abstract.html.

- [17] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, et al. Constitutional ai: Harmlessness from ai feedback, 2022. arXiv preprint. <https://doi.org/10.48550/arXiv.2212.08073>.
- [18] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. Accessed 2026-08-14 at <http://www.deeplearningbook.org>.
- [19] JoltML. Iclr 2025 test of time award: The adam optimiser and attention, 2025. Accessed 2026-08-19 at <https://joltml.com/iclr-2025/test-of-time-award/>.
- [20] Edward J Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-rank adaptation of large language models. In *The Tenth International Conference on Learning Representations*, 2022. <https://openreview.net/forum?id=nZeVKeeFYf9>.
- [21] Jack W. Rae, Sebastian Borgeaud, Trevor Cai, et al. Scaling language models: Methods, analysis insights from training gopher, 2022. arXiv preprint. <https://doi.org/10.48550/arXiv.2112.11446>.
- [22] Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, et al. Pythia: A suite for analyzing large language models across training and scaling. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 2397–2430. PMLR, 2023. <https://openreview.net/forum?id=bpRTAnJ8LW>.
- [23] Open Source Initiative. Open source ai definition, 2025. Accessed 2026-08-15 at <https://opensource.org/ai/open-source-ai-definition>.
- [24] Hugo Touvron, Thibaut Lavril, Gautier Izacard, et al. LLaMA: Open and Efficient Foundation Language Models, 2023. arXiv preprint. <https://doi.org/10.48550/arXiv.2302.13971>.
- [25] OpenAI, Josh Achiam, Steven Adler, et al. Gpt-4 technical report, 2024. arXiv preprint. <https://doi.org/10.48550/arXiv.2303.08774>.
- [26] Sarah Zhang. Exploring Fine-Tuning Techniques for Removing Tamper-Resistant Safeguards for Open-Weight LLMs. Master’s thesis, Massachusetts Institute of Technology. Department of Electrical Engineering and Computer Science, 2025. Accessed 2026-08-19 at: <https://dspace.mit.edu/handle/1721.1/159116>.

- [27] Irene Solaiman. The gradient of generative ai release: Methods and considerations. In *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency*, pages 111–122. Association for Computing Machinery, 2023. <https://doi.org/10.1145/3593013.3593981>.
- [28] Kyle O’Brien, Stephen Casper, Quentin Gregory Anthony, et al. Deep ignorance: Filtering pretraining data builds tamper-resistant safeguards into open-weight LLMs. In *The Fourteenth International Conference on Learning Representations*, 2026. <https://openreview.net/forum?id=xcf0QcTcGS>.
- [29] Nathaniel Li, Alexander Pan, Anjali Gopal, et al. The WMDP benchmark: Measuring and reducing malicious use with unlearning. In *Forty-first International Conference on Machine Learning*, volume 235, pages 28525–28550. PMLR, 2024. <https://openreview.net/forum?id=xlr6AUDuJz>.
- [30] Cameron Tice, Puria Radmard, Samuel Ratnam, et al. Alignment pretraining: AI discourse causes self-fulfilling (mis)alignment. In *Forty-third International Conference on Machine Learning*, 2026. <https://openreview.net/forum?id=9510AanYyQ>.
- [31] Domenic Rosati, Jan Wehner, Kai Williams, et al. Representation noising: A defence mechanism against harmful finetuning. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, volume 37, pages 12636–12676. Curran Associates, Inc., 2024. <https://openreview.net/forum?id=eP9auEJqFg>.
- [32] Rishub Tamirisa, Bhrugu Bharathi, Long Phan, et al. Tamper-Resistant Safeguards for Open-Weight LLMs. In *The Thirteenth International Conference on Learning Representations*, 2025. <https://openreview.net/forum?id=4FIjRodBW6>.
- [33] Xiangyu Qi, Boyi Wei, Nicholas Carlini, et al. On Evaluating the Durability of Safeguards for Open-Weight LLMs. In *The Thirteenth International Conference on Learning Representations*, 2025. <https://openreview.net/forum?id=fXJCqdUSVG>.
- [34] Minseon Kim, Jin Myung Kwak, Lama Alssum, et al. Rethinking safety in LLM fine-tuning: An optimization perspective. In *Second Conference on Language Modeling*, 2025. <https://openreview.net/forum?id=ZnOoEA2nDn>.

- [35] Zhouxiang Fang, Jiawei Zhou, and Hanjie Chen. Gr-sap: Generative replay for safety alignment preservation during fine-tuning, 2026. arXiv preprint. <https://doi.org/10.48550/arXiv.2603.10243>.
- [36] Biao Yi, Jiahao Li, Baolei Zhang, et al. Safegrad: Gradient surgery for safe llm fine-tuning. In *ICASSP 2026 - 2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 3326–3330, 2026. <https://doi.org/10.1109/ICASSP55912.2026.11463951>.
- [37] Yuntao Bai, Andy Jones, Kamal Ndousse, et al. Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback, 2022. arXiv preprint. <https://doi.org/10.48550/arXiv.2204.05862>.
- [38] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, et al. Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13484–13508. Association for Computational Linguistics, 2023. <https://doi.org/10.18653/v1/2023.acl-long.754>.
- [39] Jian Hu, Xibin Wu, Wei Shen, et al. OpenRLHF: A ray-based easy-to-use, scalable and high-performance RLHF framework. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 656–666. Association for Computational Linguistics, 2025. <https://doi.org/10.18653/v1/2025.emnlp-demos.48>.
- [40] Sagnik Mukherjee, Lifan Yuan, Pavan Jayasinha, et al. Do we need adam? surprisingly strong and sparse reinforcement learning with SGD in LLMs. In *Forty-third International Conference on Machine Learning*, 2026. <https://openreview.net/forum?id=z31fdV4WRu>.
- [41] Adam Harrison. Informatics project proposal: Visualising fine-tuning trajectories in the safety subspace., 2026.
- [42] Carl Eckart and Gale Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936. <https://doi.org/10.1007/BF02288367>.
- [43] Hugh Dalton. The measurement of the inequality of incomes. *The Economic Journal*, 30(119):348–361, 1920. <https://doi.org/10.2307/2223525>.

- [44] Niall Hurley and Scott Rickard. Comparing Measures of Sparsity. *IEEE Transactions on Information Theory*, 55(10):4723–4741, 2009. <https://doi.org/10.1109/TIT.2009.2027527>.
- [45] Guangcan Liu. Time series forecasting via learning convolutionally low-rank models. *IEEE Transactions on Information Theory*, 68(5):3362–3380, 2022. <https://doi.org/10.1109/TIT.2022.3144605>.
- [46] Andy Zou, Zifan Wang, Nicholas Carlini, et al. Universal and transferable adversarial attacks on aligned language models, 2023. arXiv preprint. <https://doi.org/10.48550/arXiv.2307.15043>.
- [47] UK AI Security Institute. Inspect AI: framework for large language model evaluations. Accessed 2026-08-20 at: https://github.com/UKGovernmentBEIS/inspect_ai.
- [48] Jiwoo Hong, Noah Lee, and James Thorne. ORPO: Monolithic preference optimization without reference model. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11170–11189. Association for Computational Linguistics, 2024. <https://doi.org/10.18653/v1/2024.emnlp-main.626>.
- [49] James Martens and Roger Grosse. Optimizing neural networks with kronecker-factored approximate curvature. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37, pages 2408–2417. PMLR, 2015. Accessed 2026-08-18 at: <https://proceedings.mlr.press/v37/martens15.html>.